

# Rising Waters Flood Prediction System

A machine-learning-based Flask web application for flood-risk classification using weather and rainfall parameters.

This report documents the implemented Rising Waters project exactly as it exists in the reviewed repository — spanning dataset preparation, model training and comparison, artifact persistence, and the deployed Flask web interface.

Python

Flask

scikit-learn

XGBoost

pandas

HTML / CSS / JavaScript

## PROJECT TEAM

Contributors to the Rising Waters Flood Prediction System

| NAME                     | ROLE        |
|--------------------------|-------------|
| Gopika Sushmitha Arigela | Team Lead   |
| P G Harshith             | Team Member |
| B Md Mahaboob Basha      | Team Member |
| Sai Tanuja Golla         | Team Member |
| Yella Naresh Kumar Reddy | Team Member |

# Table of Contents

Project Documentation — Repository Implementation Review

|           |                                                                                                                                                                                                                                                                                     |           |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>i</b>  | <b>Reader Guide</b>                                                                                                                                                                                                                                                                 | <b>1</b>  |
| <b>1</b>  | <b>Introduction</b>                                                                                                                                                                                                                                                                 | <b>2</b>  |
|           | Project Overview & Objectives · Problem Statement                                                                                                                                                                                                                                   | 2         |
| <b>2</b>  | <b>Ideation Phase</b>                                                                                                                                                                                                                                                               | <b>3</b>  |
|           | Empathy Map · Customer Journey Map · Brainstorming to Scope                                                                                                                                                                                                                         | 3         |
| <b>3</b>  | <b>Requirement Analysis</b>                                                                                                                                                                                                                                                         | <b>5</b>  |
|           | Functional & Non-Functional Requirements                                                                                                                                                                                                                                            | 5         |
| <b>4</b>  | <b>Project Design</b>                                                                                                                                                                                                                                                               | <b>6</b>  |
|           | Data Flow Diagram · Machine Learning Workflow                                                                                                                                                                                                                                       | 6         |
| <b>5</b>  | <b>Project Planning</b>                                                                                                                                                                                                                                                             | <b>8</b>  |
|           | Implementation Plan · Folder Structure                                                                                                                                                                                                                                              | 8         |
| <b>6</b>  | <b>Project Development</b>                                                                                                                                                                                                                                                          | <b>9</b>  |
|           | Dataset · EDA · Preprocessing · Scaling · Training · Persistence · Flask · Deployment                                                                                                                                                                                               | 9         |
| <b>7</b>  | <b>Testing</b>                                                                                                                                                                                                                                                                      | <b>15</b> |
| <b>8</b>  | <b>Results</b>                                                                                                                                                                                                                                                                      | <b>16</b> |
|           | Application Screenshots                                                                                                                                                                                                                                                             | 17        |
| <b>9</b>  | <b>Advantages &amp; Limitations</b>                                                                                                                                                                                                                                                 | <b>19</b> |
| <b>10</b> | <b>Future Scope &amp; Conclusion</b>                                                                                                                                                                                                                                                | <b>20</b> |
| <b>11</b> | <b>References</b>                                                                                                                                                                                                                                                                   | <b>21</b> |
| <b>12</b> | <b>Appendix</b>                                                                                                                                                                                                                                                                     | <b>22</b> |
|           | Page numbers refer to the project documentation body. This report is a presentation-focused redesign of the original documentation and preserves all verified technical content.                     A. Environment & Execution · B. Runtime Contracts · C. Source Review Checklist | 22        |



## Document Scope

This report records the implemented Rising Waters project as it exists in the reviewed repository. It deliberately distinguishes source-backed facts from design artefacts. The architecture, data-flow, empathy, journey, brainstorming and workflow diagrams are original representations created for this report; they do not claim components beyond the repository.

**BOUNDARY OF THIS REPORT**

The report does not report a fixed accuracy, probability, database, user account, prediction history, third-party weather API, or cloud provider feature unless it is present in the reviewed implementation. Supplied third-party sample materials informed only the requested report organization and visual direction — never the technical content.

## Evidence Reviewed

| Area             | Verified Evidence                                          |
|------------------|------------------------------------------------------------|
| Application      | app.py, four Jinja templates, static CSS and JavaScript    |
| Machine Learning | Notebook, src/train_models.py, transform.save, floods.save |
| Data             | data/flood_dataset.xlsx                                    |
| Delivery         | README live URL and Git remote                             |
| Visual Evidence  | Four PNG screenshots in screenshots/                       |

Table i. Source materials used to prepare this report.

# 1 Introduction

---

Flood events can require timely interpretation of weather and rainfall conditions. Rising Waters is a compact machine-learning-based web application that accepts a fixed set of meteorological and rainfall parameters and returns one of two classification outcomes.

The implemented system is a supervised binary classification workflow. It begins with an Excel dataset, documents exploratory analysis and preprocessing in a Jupyter notebook, trains and compares four classifiers through a Python module, persists a scaler and selected model, and serves prediction pages through Flask.

Flood Risk Detected

No Flood Risk Detected

## INTENTIONAL SCOPE

The project is intentionally focused: it does not collect live weather data, store submissions, expose an API, retain user accounts, or display a prediction probability.

## Problem Statement

---

Provide a browser-based way to submit the ten weather and rainfall features used by the trained model, apply the persisted preprocessing transformation consistently, and present the resulting flood-risk classification.

## Project Overview and Objectives

---

The project combines an exploratory notebook and a deployable Flask interface. The notebook handles data review and preparation; `src/train_models.py` handles model training, evaluation output, comparison, selection and persistence; `app.py` handles runtime loading, form validation, scaling and prediction.

### Objectives

- Load and examine the Excel flood dataset using pandas.
- Visualize individual numerical distributions and numerical correlations in the notebook.
- Check missing values, cap IQR outliers without deleting rows, and prepare the target split.
- Use a training/test split of 80/20 with `random_state=42` and `StandardScaler`.
- Train Decision Tree, Random Forest, K-Nearest Neighbors and XGBoost classifiers.
- Select the highest-accuracy model, preferring XGBoost when the best accuracy is tied.
- Expose the persisted model in a simple Flask form workflow.

## 2 Ideation Phase

The implementation can be understood as a narrow decision-support flow rather than a comprehensive flood-management platform. The core question is: given the recorded feature values expected by the model, which of the two stored classes should be returned?

The ideation was translated into four implementation tracks shown below. Each track corresponds to a real repository area.

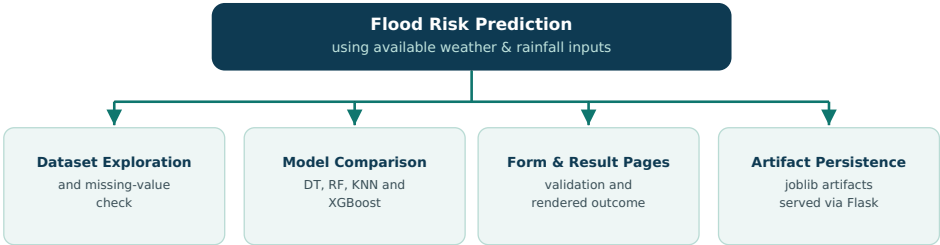


Figure 1. Original ideation map based on the implemented data, model, interface and delivery tracks.

**SCOPE BOUNDARY**

The source code contains no notification service, geospatial map, database schema, authentication system, or historical prediction dashboard.

### Empathy Map

This original empathy map describes the immediate interaction supported by the current interface. It is not user research evidence and it does not imply personas or capabilities not implemented in the application.

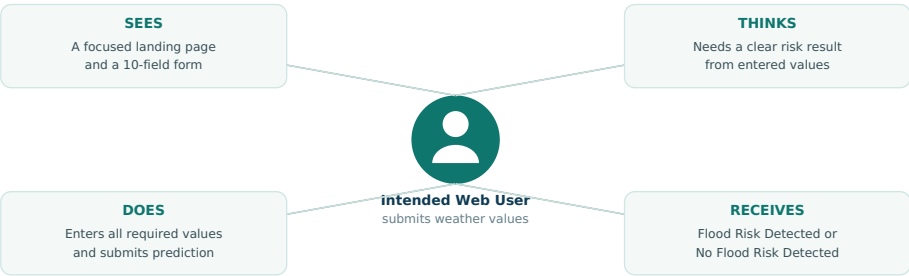


Figure 2. Original empathy map for the implemented prediction-form journey.

## 2 Ideation Phase (continued)

### Customer Journey Map

The browser journey starts at the landing page and ends at a binary result page. The journey below follows the actual routes and templates in app.py.

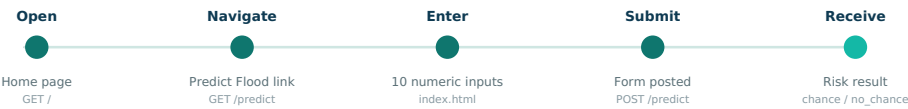


Figure 3. Original customer journey map for the implemented browser flow.

| Journey Stage | Implementation Evidence                           |
|---------------|---------------------------------------------------|
| Open          | GET / renders templates/home.html                 |
| Navigate      | Predict Flood link opens GET /predict             |
| Enter         | index.html exposes ten numeric inputs             |
| Submit        | Form posts to POST /predict                       |
| Receive       | chance.html for class 1; no_chance.html otherwise |

Table 1. Route-level journey trace.

### Brainstorming to Scope

The project scope is deliberately achieved through reusable components rather than a large platform. The data track is isolated in the notebook; the training track is isolated in src/train\_models.py; the application track is isolated in app.py and the Jinja templates.

| Decision            | Implemented Response               | Out-of-Scope Consequence           |
|---------------------|------------------------------------|------------------------------------|
| Single dataset      | Read a local XLSX file             | No live data ingestion             |
| Binary output       | Return one of two result templates | No probability visualization       |
| Persisted artifacts | Use joblib at startup              | No online training during requests |
| Static form         | Ten named numeric inputs           | No dynamic feature schema in UI    |

Table 2. Scope decisions observable in the repository.

### 3 Requirement Analysis

Functional requirements are derived from the implemented application and training module, not from reference material. The table identifies the concrete behavior that is present in the repository.

#### Functional Requirements

| ID    | Requirement Implemented              | Verification Source                       |
|-------|--------------------------------------|-------------------------------------------|
| FR-01 | Load flood_dataset.xlsx              | Notebook and train_models.py              |
| FR-02 | Explore and visualize numerical data | FloodPrediction.ipynb                     |
| FR-03 | Cap numerical IQR outliers           | FloodPrediction.ipynb                     |
| FR-04 | Train and compare four classifiers   | src/train_models.py                       |
| FR-05 | Persist scaler and selected model    | models/transform.save; models/floods.save |
| FR-06 | Accept ten form inputs and predict   | app.py and index.html                     |
| FR-07 | Show class-specific result page      | chance.html; no_chance.html               |

Table 3. Functional requirements present in the implementation.

#### Non-Functional Requirements

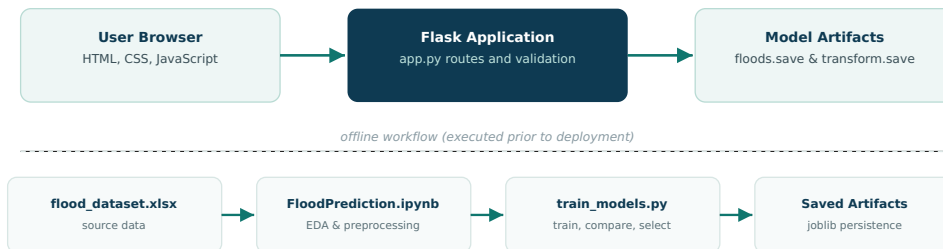
The following characteristics are evidenced by code structure and interface assets. They are not benchmark claims.

| Quality Area         | Implemented Approach                                                                          |
|----------------------|-----------------------------------------------------------------------------------------------|
| Maintainability      | Path constants, small reusable functions, docstrings and comments in Python modules.          |
| Consistency          | FEATURE_NAMES in app.py fixes the runtime DataFrame order to the training feature order.      |
| Input Validation     | JavaScript checks blank fields; Flask validates values and catches conversion/runtime errors. |
| Usability            | Responsive CSS changes the form grid to one column below 720px.                               |
| Reproducibility      | Fixed train/test configuration: test_size=0.20 and random_state=42.                           |
| Deployment Readiness | requirements.txt includes gunicorn and README documents a Render URL.                         |

Table 4. Non-functional implementation characteristics.

## 4 Project Design

Rising Waters has two connected paths: an offline machine-learning path that creates artifacts and an online Flask path that consumes them. Keeping the paths separate ensures a prediction request does not retrain a model.

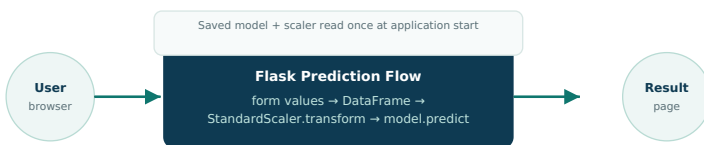


**Figure 4.** Original architecture diagram showing only components present in the repository.

The training module reads the same Excel dataset used by the notebook. At application startup, app.py loads both persisted artifacts once, rather than loading them on every request.

### Data Flow Diagram

The data-flow diagram models the runtime request. It contains no database or history store because neither exists in the source code. The model artifacts are read-only dependencies at prediction time.



**Figure 5.** Original Level-0 data flow for an online prediction request.

#### ERROR HANDLING

On error, POST /predict re-renders the input template with an error message and HTTP status 400. The templates reviewed do not display an explicit error placeholder, but the server supplies the context.



## 4 Project Design (continued)

### Machine Learning Workflow

The notebook defines the dataset preparation sequence. The training module subsequently loads the dataset, re-creates X and y, applies the persisted scaler to a fresh 80/20 split, trains models and selects the best accuracy.



**Figure 6.** Original pipeline diagram reflecting notebook and training-module stages.

The notebook saves the fitted StandardScaler to ../models/transform.save. The training module saves the selected model to models/floods.save.

## 5 Project Planning

The repository represents a staged implementation rather than a project-management system. The notebook titles label Epics 1 through 3, while `src/train_models.py` describes Epic 4. The Flask application and frontend provide the deployed interface.

| Phase           | Primary Implementation Output                          | Repository Location                                                  |
|-----------------|--------------------------------------------------------|----------------------------------------------------------------------|
| Data Collection | Excel file loaded with pandas                          | <code>data/</code> ; <code>FloodPrediction.ipynb</code>              |
| Analysis        | Exploration, distributions and correlations            | <code>FloodPrediction.ipynb</code>                                   |
| Preprocessing   | Missing-value analysis, IQR capping, split and scaling | <code>FloodPrediction.ipynb</code>                                   |
| Model Building  | Training, metrics, comparison and persistence          | <code>src/train_models.py</code>                                     |
| Application     | Flask routes and responsive pages                      | <code>app.py</code> ; <code>templates/</code> ; <code>static/</code> |
| Deployment      | Live URL documented                                    | <code>README.md</code>                                               |

Table 5. Implementation-aligned project plan.

### Project Folder Structure

The project is intentionally small. The folders below are the active implementation areas reviewed for this report.

```
rising-waters-flood-prediction/  
├─ app.py  
├─ data/flood_dataset.xlsx  
├─ models/floods.save  
├─ models/transform.save  
├─ notebooks/FloodPrediction.ipynb  
├─ src/train_models.py  
├─ templates/{home,index,chance,no_chance}.html  
├─ static/css/main.css  
├─ static/js/main.js  
├─ screenshots/*.png  
├─ requirements.txt  
└─ README.md
```

| Folder / File             | Responsibility                                                |
|---------------------------|---------------------------------------------------------------|
| <code>data/</code>        | Source Excel dataset                                          |
| <code>models/</code>      | Persisted <code>StandardScaler</code> and selected classifier |
| <code>notebooks/</code>   | EDA and preprocessing workflow                                |
| <code>src/</code>         | Reusable model training and comparison code                   |
| <code>templates/</code>   | Jinja HTML views                                              |
| <code>static/</code>      | CSS visual system and client-side validation                  |
| <code>screenshots/</code> | Existing application screenshots                              |
| <code>app.py</code>       | Flask request handling and inference                          |

Table 6. Reviewed project structure.

## 6 Project Development

### Dataset Information

The project reads `data/flood_dataset.xlsx`. It contains 115 records and 11 columns: ten features and the binary target column `flood`. Every reviewed column is numerical; the notebook therefore reports that label encoding is not required.

| Column                                     | Type                 | Role                        |
|--------------------------------------------|----------------------|-----------------------------|
| Temp, Humidity, Cloud Cover                | <code>int64</code>   | Weather inputs              |
| ANNUAL, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec | <code>float64</code> | Rainfall-related inputs     |
| avgjune, sub                               | <code>float64</code> | Additional numerical inputs |
| flood                                      | <code>int64</code>   | Binary target               |

Table 7. Dataset schema verified from `flood_dataset.xlsx`.

#### CLASS BALANCE

The target distribution is 99 records for class 0 and 16 records for class 1. This report records the class count as data context, not as a claim of model effectiveness.

### Exploratory Data Analysis

The notebook's exploration section displays `df.head()`, `df.shape`, `df.columns`, `df.info()`, `df.describe()`, `df.dtypes` and `df.isnull().sum()`. Its descriptive-analysis section adds `df.describe(include='all')`, `df.nunique()` and a printed dataset summary.

Univariate analysis automatically identifies numerical columns and excludes `flood`. It loops through features to create a Seaborn histogram with KDE and a Seaborn boxplot for each. Multivariate analysis computes a numerical correlation matrix, shows a coolwarm annotated heatmap, reports correlations with `flood` and conditionally creates scatter plots for `ANNUAL`, `Jun-Sep` and `Humidity` against `flood`.

#### INTERPRETATION BOUNDARY

The notebook provides exploratory visuals; it does not persist charts as application output, apply feature selection, or create a dashboard.

### Data Preprocessing — Missing Values

The preprocessing portion of the notebook checks `df.isnull().sum()` and `df.isnull().any()`, then prints a message about whether missing values exist. For the reviewed dataset, each of the 11 columns has zero missing values. Consequently, no missing-value treatment is required by this implementation.

| Check                             | Verified Result    |
|-----------------------------------|--------------------|
| Dataset rows                      | 115                |
| Dataset columns                   | 11                 |
| Missing values across all columns | 0                  |
| Categorical columns               | None detected      |
| Target column                     | <code>flood</code> |

Table 8. Dataset checks evaluated from the supplied Excel file.

This is an important distinction: the code checks for missing values, but it does not impute them because the actual dataset did not require imputation.

### Data Preprocessing — Outlier Handling

For each numerical feature except `flood`, the notebook calculates the first and third quartiles, derives the interquartile range (IQR), computes lower and upper limits, counts values outside these limits, and

replaces extreme values through pandas clip()).

Capping is used instead of row deletion. This retains the dataset's 115 rows while constraining extreme feature values to their IQR thresholds. The notebook prints the count of capped values for every feature.

| Step | Notebook Operation                                                              |
|------|---------------------------------------------------------------------------------|
| 1    | Select numeric columns excluding flood                                          |
| 2    | Compute Q1, Q3 and IQR for each feature                                         |
| 3    | Set lower_bound = $Q1 - 1.5 \times IQR$ and upper_bound = $Q3 + 1.5 \times IQR$ |
| 4    | Count values below / above bounds                                               |
| 5    | Replace values with <code>series.clip(lower, upper)</code>                      |

Table 9. IQR-based capping logic documented in the notebook.

### Feature Preparation and Splitting

The notebook detects categorical columns using `select_dtypes(include=['object','category'])`. When such columns exist, it applies `LabelEncoder` only to those columns. For the reviewed dataset, it prints: "The dataset contains only numerical features. Therefore, label encoding is not required."

```
X = dataset.drop('flood', axis=1)
y = dataset['flood']
```

The notebook then uses `train_test_split` with `test_size=0.20` and `random_state=42`, and prints the shapes of `X`, `y`, `X_train`, `X_test`, `y_train` and `y_test`. The training module repeats this split configuration when preparing models.

### Feature Scaling

The notebook imports `StandardScaler`, fits it on `X_train` and transforms both `X_train` and `X_test`. It saves the fitted scaler using `joblib` at `../models/transform.save`.

At training time, `prepare_train_test_data()` loads the same scaler from `models/transform.save` and calls `scaler.transform` on the freshly split train and test features. At application time, `app.py` applies `scaler.transform` to a one-row `DataFrame` before calling the classifier. The model and interface therefore use the same feature order and the same persisted scaling configuration.

| Stage           | Feature Scaling Behavior                                                               |
|-----------------|----------------------------------------------------------------------------------------|
| Notebook        | Fit <code>StandardScaler</code> on training features; save <code>transform.save</code> |
| Training module | Load <code>transform.save</code> ; transform new split arrays                          |
| Flask request   | Create ordered <code>DataFrame</code> ; transform with loaded scaler                   |

Table 10. Scaling artifact lifecycle.

### Model Training

`src/train_models.py` exposes reusable functions for Decision Tree, Random Forest, K-Nearest Neighbors and XGBoost. Each function fits a classifier, predicts the test data, prints accuracy, a confusion matrix and a classification report, then returns (model, accuracy).

| Function                     | Classifier Instantiation                                           |
|------------------------------|--------------------------------------------------------------------|
| <code>decision_tree()</code> | <code>DecisionTreeClassifier(random_state=42)</code>               |
| <code>random_forest()</code> | <code>RandomForestClassifier(random_state=42)</code>               |
| <code>knn()</code>           | <code>KNeighborsClassifier()</code>                                |
| <code>xgboost_model()</code> | <code>XGBClassifier(random_state=42, eval_metric='logloss')</code> |

Table 11. Classifiers configured in `src/train_models.py`.

The source code does not set a class-weight strategy, probability threshold, hyperparameter-search routine, cross-validation loop or custom feature engineering routine.

### Model Evaluation and Comparison

`compare_models()` evaluates all four training functions and builds a pandas table with Model Name and Accuracy. It prints the comparison table, identifies the maximum accuracy and saves the selected classifier.

#### TIE BEHAVIOR

The selection logic is explicit: when XGBoost is among the models tied at the highest accuracy, XGBoost is selected. Otherwise, the first tied result is selected. This report does not publish a static accuracy because the repository stores no executed training log or metrics report; evaluation values are printed when the training module runs.

The saved `floods.save` artifact contains an `XGBClassifier` signature. The code also gives XGBoost priority in any highest-accuracy tie. The project documentation and source therefore identify XGBoost as the final deployment model.

The training script does not force XGBoost to replace a unique higher-scoring alternative. It saves the model selected by its accuracy comparison, with XGBoost as the deterministic tie-breaker.

| Artifact                           | Verified Role                                               |
|------------------------------------|-------------------------------------------------------------|
| <code>models/floods.save</code>    | Persisted final classifier used by <code>app.py</code>      |
| <code>models/transform.save</code> | Persisted <code>StandardScaler</code> used before inference |
| <code>src/train_models.py</code>   | Creates and compares candidate classifiers                  |

Table 12. Final-model evidence chain.

### Model Persistence

joblib is used for persistence. The notebook saves transform.save after fitting StandardScaler. The training module creates the models directory if needed and writes the selected model to floods.save. app.py resolves both paths relative to the project root and loads each artifact once during startup.

#### RUNTIME SAFETY

If an artifact does not exist or cannot be loaded, app.py records the startup exception. A subsequent prediction request raises a controlled runtime error and returns the form template with status 400 rather than attempting inference with missing artifacts. The persisted objects are local repository files — no artifact registry, remote storage system, or model version service is present in the reviewed code.

### Flask Application Workflow

The Flask application creates three primary behaviors: render the landing page, render the prediction form and process a submitted form. A fourth /home route redirects to /. This runtime relationship between the browser, Flask routes and the persisted artifacts is the same one shown in **Figure 4**. The route table below follows the source exactly.

| Route    | Method | Result                                     |
|----------|--------|--------------------------------------------|
| /        | GET    | Render home.html                           |
| /predict | GET    | Render index.html                          |
| /predict | POST   | Validate, scale, predict and render result |
| /home    | GET    | Redirect to /                              |

Table 13. Flask routes in app.py.

### Inference Request Sequence

For POST /predict, app.py loops over FEATURE\_NAMES, reads each request.form value, rejects missing / blank values, converts each value to float, creates a one-row pandas DataFrame in exact feature order, scales it and predicts class 0 or 1.

If int(prediction) == 1, the response renders chance.html. Any other prediction renders no\_chance.html. Exceptions are caught and return index.html with a 400 response.

#### ORDERED FEATURE NAMES

Temp, Humidity, Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, avgjune, sub

Table 14. Feature order hard-coded in app.py and reflected in index.html.

### Frontend Pages

The frontend uses server-rendered Jinja templates and one shared stylesheet. It does not use a frontend framework or separate API client.

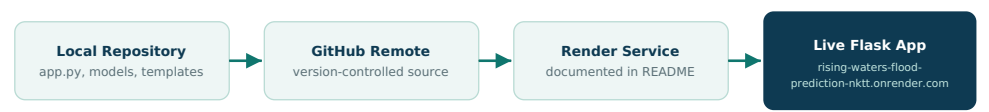
| Page           | Purpose               | Key Implementation                                      |
|----------------|-----------------------|---------------------------------------------------------|
| home.html      | Landing page          | Project title, short description and Predict Flood link |
| index.html     | Prediction form       | Ten numeric inputs; POST /predict                       |
| chance.html    | Positive class result | Flood Risk Detected and Predict Again link              |
| no_chance.html | Negative class result | No Flood Risk Detected and Predict Again link           |
| main.js        | Client validation     | Blocks submission if a required input is blank          |
| main.css       | Presentation          | Responsive card layout, colors and hover states         |

Table 15. Implemented presentation layer.

## Deployment and Repository

The README documents a live Render application at `rising-waters-flood-prediction-nktt.onrender.com` and notes that the free tier can take up to 60 seconds to wake after idling. `requirements.txt` includes `gunicorn`, which supports a production WSGI deployment.

The configured Git remote is `github.com/harshithnayak07/rising-waters-flood-prediction`. This report treats the remote and README URL as deployment evidence; it does not infer a `render.yaml`, build command, GitHub Actions workflow, or cloud architecture because these are not in the reviewed repository.



**Figure 8.** Original deployment path based on the configured GitHub remote and README Render URL.



## 7 Testing

The repository does not contain an automated test suite. Accordingly, this report does not claim unit-test coverage, load-test results, or browser-test evidence. The testable behavior was reviewed from the code paths, data schema, saved artifact signatures and existing UI screenshots.

| Test Area             | Expected Behavior from Implementation                      |
|-----------------------|------------------------------------------------------------|
| Dataset availability  | data/flood_dataset.xlsx exists with expected target column |
| Artifact availability | transform.save and floods.save exist                       |
| Feature order         | app.py constructs DataFrame with FEATURE_NAMES             |
| Blank input           | main.js blocks client submit; app.py also rejects it       |
| Non-numeric input     | Float conversion raises a handled error                    |
| Class 1               | chance.html rendered                                       |
| Other class           | no_chance.html rendered                                    |

Table 16. Source-derived verification matrix.

### Testing Evidence and Constraints

Static evidence confirms that app.py has the required routes, model/scaler path handling and error boundary, and that the template form names match the exact ten runtime feature names. The project screenshot set captures the landing, form and both result states.

#### WHAT IS NOT CLAIMED

- No fixed accuracy, since metrics are printed at training time and no metrics log is persisted.
- No production uptime, latency, concurrency, or Render health claim.
- No trained-model calibration, fairness, drift, or robustness measurement.
- No automated regression test execution, because no test files are present.

## 8 Results

The completed repository contains a complete local workflow from Excel input to Flask-rendered classification result. It includes a notebook with EDA and preprocessing, model-training code for four classifiers, a persisted StandardScaler, a persisted XGBoost classifier artifact, a form-driven Flask interface, and a documented Render deployment URL.

The functional result presented by the user interface is categorical. The two result states are visibly represented by the screenshots on the following page and correspond to the two templates returned by app.py.

| Result Area         | Implemented Output                                                      |
|---------------------|-------------------------------------------------------------------------|
| Data review         | Notebook displays structure, statistics, distributions and correlations |
| Preprocessing       | Missing-value check, IQR capping, split and scaling                     |
| Training            | Printed evaluation output per model plus comparison table               |
| Deployment artifact | floods.save and transform.save                                          |
| User interface      | Binary flood-risk result page                                           |

Table 17. Implementation results without unverified performance metrics.

The following images are the existing files from screenshots/. They are embedded as evidence only; they have not been recreated or modified.

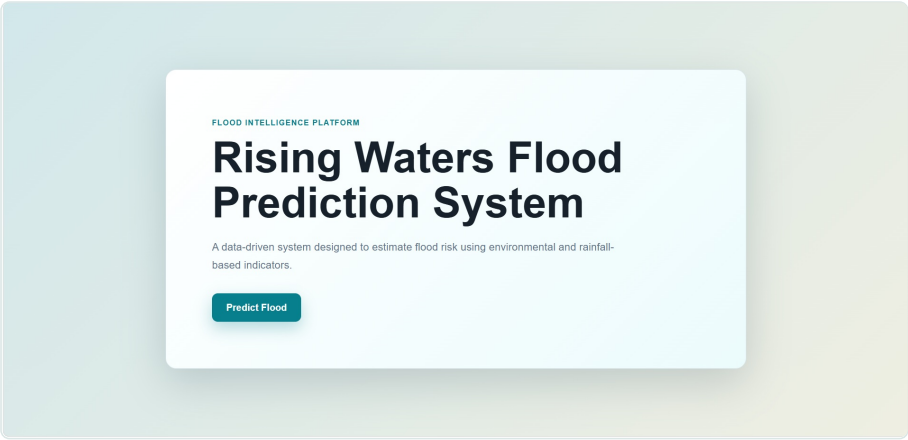
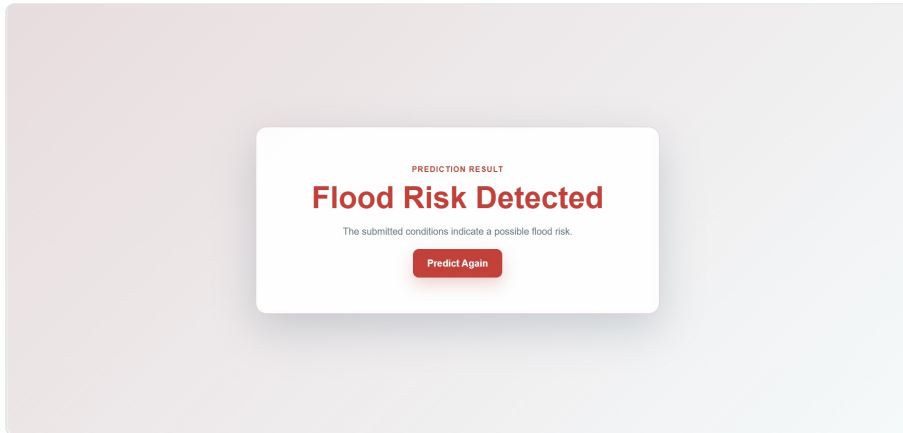


Figure 9. Existing home page screenshot (screenshots/home\_page.png).

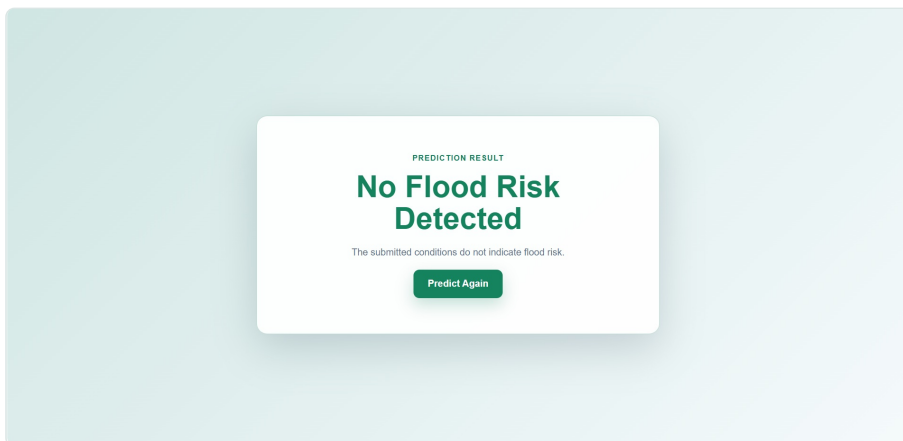
A screenshot of a web application's prediction form. The form has a light blue and white color scheme. At the top, it says "PREDICTION FORM". Below that, the main heading is "Flood Risk Prediction". Underneath the heading, there is a subtext: "Enter the environmental and rainfall feature values to estimate flood risk." The form contains several input fields arranged in two columns. The left column includes: "Temperature" (value: 20), "Cloud Cover" (value: 45), "Jan-Feb Rainfall" (value: 60.25), "Jun-Sep Rainfall" (value: 300.44), and "Average June Rainfall" (value: 2000.00). The right column includes: "Humidity" (value: 100), "Annual Rainfall" (value: 25.22), "Mar-May Rainfall" (value: 2001.00), "Oct-Dec Rainfall" (value: 50.22), and "Sub-Division Value" (value: 2001.00). At the bottom of the form, there is a teal button labeled "Submit Prediction".

Figure 10. Existing prediction-form screenshot (screenshots/prediction\_form.png).

The two result templates use distinct visual treatments: red for a detected flood risk and green for no flood risk. Both provide a Predict Again link to the GET /predict form.



**Figure 11.** Existing flood-risk result screenshot (*screenshots/flood\_result.png*).



**Figure 12.** Existing no-flood-risk result screenshot (*screenshots/no\_flood\_result.png*).

## 9 Advantages & Limitations

---

### Advantages

---

- Clear separation between exploration/preprocessing, training, persisted artifacts and runtime inference.
- A single ordered feature list protects inference DataFrame ordering.
- Model and scaler load once at application startup instead of per request.
- IQR capping retains all dataset rows rather than deleting observations.
- The model-comparison function applies a deterministic XGBoost preference in a tie.
- Client-side and server-side blank-value handling both exist.
- The responsive CSS reduces the prediction form to one column on smaller screens.

#### NOTE

These are implementation characteristics, not claims of domain-level flood forecasting reliability.

### Disadvantages and Limitations

---

- The dataset has 115 records and is imbalanced: 99 class-0 and 16 class-1 records.
- The form accepts manually entered values only; it does not retrieve live weather information.
- The application returns a binary class, not probability or confidence information.
- The form has no field-level range validation beyond numeric conversion and empty-value checks.
- No test suite, authentication, database, or prediction-history storage is included.
- The training script reads the source Excel file; it does not save a separate fully preprocessed dataset.
- No model metrics are persisted for audit or display after training.

## 10 Future Scope & Conclusion

The following items are not implemented. They are presented as future work because they would build on, rather than misrepresent, the current architecture.

| Potential Enhancement          | Why It Follows the Current Project                                              |
|--------------------------------|---------------------------------------------------------------------------------|
| Persist training metrics       | Would allow the comparison output to be reported and audited after runs.        |
| Automated tests                | Would verify Flask routes, form errors and artifact compatibility.              |
| Input range validation         | Could prevent values outside expected weather/rainfall ranges.                  |
| Live data integration          | Could replace manual form entry with a verified weather source.                 |
| Model monitoring               | Could track data changes and re-evaluation needs over time.                     |
| Authentication and persistence | Could support controlled access and saved prediction records.                   |
| Cloud deployment configuration | Could make the documented Render deployment reproducible from repository files. |

Table 18. Clearly separated future scope items.

### Conclusion

Rising Waters is a complete compact flood-risk classification project built around a local Excel dataset, a Jupyter-based analysis and preprocessing workflow, model comparison in Python, persisted StandardScaler and XGBoost artifacts, and a Flask web application. Its deployed interface accepts ten named input features, applies the saved scaler and returns a binary risk page.

The project's strongest architectural property is its separation of offline preparation/training from online inference. Its principal limitations are equally clear in the code: a small imbalanced dataset, no saved metric history, no automated tests, and no live-weather integration. This report has preserved those boundaries so the documentation remains faithful to the submitted implementation.

## 11 References

---

1. Rising Waters source repository: [github.com/harshithnayak07/rising-waters-flood-prediction](https://github.com/harshithnayak07/rising-waters-flood-prediction) (configured Git remote reviewed for this report).
2. Project README.md: local installation, training and running instructions; includes the documented Render URL.
3. app.py: Flask routes, artifact loading, feature ordering, validation and prediction behavior.
4. src/train\_models.py: dataset loading, train/test split, StandardScaler loading, model functions, comparison and persistence.
5. notebooks/FloodPrediction.ipynb: data collection, EDA, preprocessing, feature preparation and scaling workflow.
6. data/flood\_dataset.xlsx: source dataset schema and counts used in this report.
7. templates/, static/ and screenshots/: implementation and UI evidence.

### REFERENCE MATERIAL USED FOR PRESENTATION ONLY

Supplied third-party sample report materials and reference images were used only to understand report-structure expectations; they are not cited as sources for project behavior and no content, text, diagrams, or tables were copied from them.

## 12 Appendix A — Environment & Execution

requirements.txt pins the libraries below. The project README gives the corresponding local commands: create a virtual environment, install requirements, run `python src/train_models.py` if model artifacts are unavailable, then run `python app.py`.

| Package      | Pinned Version |
|--------------|----------------|
| pandas       | 2.3.3          |
| numpy        | 2.2.6          |
| matplotlib   | 3.10.9         |
| seaborn      | 0.13.2         |
| scikit-learn | 1.7.2          |
| xgboost      | 3.2.0          |
| Flask        | 3.1.3          |
| joblib       | 1.5.3          |
| openpyxl     | 3.1.5          |
| unicorn      | 26.0.0         |

Table 19. Dependencies listed in requirements.txt.

## Appendix B — Key Runtime Contracts

The contracts below must remain compatible for prediction requests to work.

| Contract                | Requirement                                                                                                                                        |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset to training     | Target column is exactly flood; remaining ten columns are input features.                                                                          |
| Training to persistence | The scaler is stored at <code>models/transform.save</code> and model at <code>models/floods.save</code> .                                          |
| Persistence to Flask    | <code>app.py</code> uses the ordered <code>FEATURE_NAMES</code> list and applies <code>scaler.transform</code> before <code>model.predict</code> . |
| Template to Flask       | <code>index.html</code> input name attributes match the feature list exactly.                                                                      |

Table 20. Compatibility contracts across the implemented system.

### MAINTENANCE NOTE

Maintaining these names and ordering is essential. Changing an input field name, reordering features without retraining, or replacing only one artifact can break or invalidate inference.



## C Appendix C — Source Review Checklist

Completed review basis for this documentation.

| Reviewed Item   | Finding                                                                            |
|-----------------|------------------------------------------------------------------------------------|
| Dataset         | 115 × 11 Excel dataset; no missing values; numerical columns only                  |
| Notebook        | 38 cells; Epics 1–3 implemented                                                    |
| Training module | Four model functions; comparison and XGBoost tie preference                        |
| Artifacts       | XGBClassifier signature in floods.save; StandardScaler signature in transform.save |
| Flask           | Home, form, prediction and redirect routes                                         |
| Frontend        | Four templates, shared CSS, basic JavaScript validation                            |
| Deployment docs | README Render URL; GitHub origin remote                                            |
| Screenshots     | Four existing PNG files embedded in report                                         |

Table 21. Final source-review checklist.